

LARA

Latency-Aware Rerank-then-Allocate Architecture

for Resource-Constrained Retrieval-Augmented Generation

Adaptive Retrieval · Quality-Gated Reranking · Latency Governance

Tier-Based Context Safety · Sliding Window Refinement

Sonu Kumar

Founder, NPMAI ECOSYSTEM

Software Developer | AI Developer | TEDx Speaker

Student — Allen Career Institute & Disha Delphi Public School, Kota

sonuramashishnpm@gmail.com | npmai.netlify.app

Abstract

If you build a RAG system today and someone asks why you set $k=4$, the honest answer is: because that is the default. Nobody proved it was right. This is the fixed- k problem — a fundamental design flaw in standard Retrieval-Augmented Generation pipelines that produces noise injection on small corpora, context overflow on large corpora, and middle context loss in any setting where too many chunks reach the language model simultaneously.

This paper presents **LARA** — Latency-Aware Rerank-then-Allocate Architecture — a four-phase adaptive retrieval pipeline. LARA uses ANN-based candidate capture in $O(\log N)$ time, cross-encoder reranking with a quality threshold of 0.3 to eliminate noise, a Latency Governor with a formal mathematical correctness proof, and a Tier-Based Safety Gate with Sliding Window Refinement that derives batch size from the model's context window and Liu et al.'s (2024) middle context loss findings. LARA is deployed in the npmai Python package with over 700,000 installations. The paper also proposes an ML extension — Index-Structure-Aware K Prediction — that reads the internal structure of the vector index to predict optimal candidate pool size. Full benchmarks are ongoing and will be reported in a companion paper.

Keywords: Retrieval-Augmented Generation, Adaptive Retrieval, Cross-Encoder Reranking, Latency-Aware Systems, Middle Context Loss, Context Window Safety, Index-Structure-Aware K Prediction, NPM AI ECOSYSTEM

Table of Contents

1. Introduction
2. Background and Related Work
 - 2.1 The Fixed-K Problem
 - 2.2 Middle Context Loss
 - 2.3 Existing Adaptive RAG Systems
 - 2.4 Two-Stage Retrieve-then-Rerank
3. The LARA Architecture
 - 3.1 Phase 1 & 2 — ANN Indexing and Candidate Capture
 - 3.2 Phase 3 — The Quality Gate
 - 3.3 Phase 4 — The Latency Governor
 - 3.4 Phase 5 — Tier Safety Gate and Sliding Window
4. Theoretical Analysis
 - 4.1 $O(\log N)$ Scaling Argument
 - 4.2 Latency Governor Correctness Proof
 - 4.3 Middle Context Loss Prevention
 - 4.4 Deriving the Safety Tier Table
5. ML Extension — Index-Structure-Aware K Prediction
 - 5.1 The Problem With Fixed Candidate Pool Size
 - 5.2 Index Structure as Distribution Proxy
 - 5.3 How Each Index Type Is Read
 - 5.4 Model Architecture and Training
6. Implementation
7. Experimental Setup
8. Discussion
9. Conclusion
- References

1. Introduction

There is a question nobody in the RAG community asks loudly enough: why $k=4$? The default retrieval parameter in LangChain, LlamaIndex, and most RAG frameworks is $k=4$. It has been $k=4$ since the early days of the paradigm and it remains $k=4$ in most production systems today. The honest answer for why: nobody proved it was right. It was chosen as a reasonable default and it stuck.

This matters because k determines everything downstream — what the language model sees, how much noise it receives, how complete its context is, and whether it can answer the question at all. Setting it arbitrarily and universally is not a minor engineering choice. It is the central failure of the standard RAG paradigm, and it produces three distinct categories of harm.

First, when the corpus is small — say 5 total chunks — retrieving $k=4$ forces the system to surface marginally relevant or completely irrelevant material, injecting noise directly into the context and increasing hallucination risk. Second, when the corpus is large — hundreds or thousands of chunks — retrieving $k=4$ leaves the vast majority of relevant information unretrieved, producing incomplete answers. Third, even when k happens to be the right size, passing all chunks to the LLM in one call causes middle context loss — Liu et al. (2024) showed that LLM performance follows a U-shaped curve with respect to the position of relevant content in long contexts.

LARA addresses all three failures simultaneously through four principled phases. I also want to be upfront about what this paper does and does not claim. The architecture is fully specified and deployed. The Latency Governor has a formal correctness proof. The Safety Tier table is derived from Liu et al.'s empirical findings. The ML extension is a proposed architecture — novel and theoretically motivated — with training and benchmarks planned for the companion empirical paper. I state all of this clearly in each relevant section.

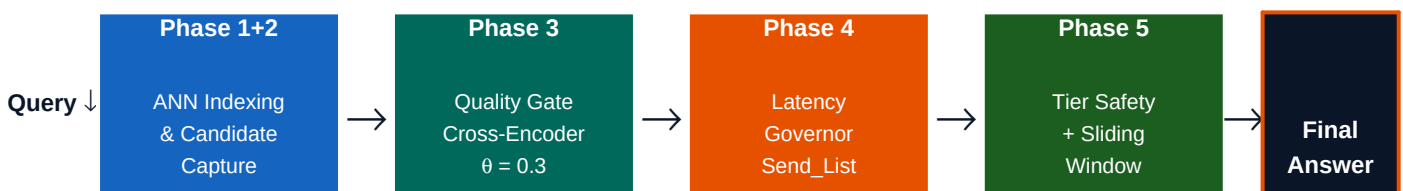


Figure 1: The complete LARA pipeline — four phases from query to final answer.

2. Background and Related Work

2.1 The Fixed-K Problem

The original RAG formulation (Lewis et al., 2020) retrieves a fixed number of documents k from an external knowledge source. This design reflected the context window constraints of early language models — typically 512 to 2048 tokens — and has persisted as a default despite the significant expansion of context windows in modern models.

RankRAG (Chen et al., 2024) explicitly acknowledges the fundamental accuracy-noise tradeoff: smaller k misses relevant chunks while larger k introduces irrelevant context. Their solution — fine-tuning the LLM for integrated ranking and generation — addresses the symptom rather than the cause. CAR (2024) detects transition points in similarity score distributions to determine optimal retrieval depth. Adaptive- k (Taguchi et al., 2025) selects retrieval count based on similarity distributions. DynamicRAG (2025) uses reinforcement learning to dynamically determine both ranking and number of retrieved documents. None address the compound problem of simultaneously optimising retrieval count, semantic quality, latency budget, and context window utilisation in a single unified pipeline.

2.2 Middle Context Loss

Liu et al. (2024) published a landmark finding in TACL: LLM performance on multi-document QA degrades significantly when relevant information is positioned in the middle of long input contexts. Performance follows a U-shaped curve — highest at the beginning and end of context. Critically, this degradation scales with absolute context length, not merely with the proportion of the context window used. A 30,000-token context in a 128K window suffers more severe degradation than a 3,000-token context in a 4K window.

LARA is, to my knowledge, the first RAG architecture to treat middle context loss as an explicit design constraint rather than an acknowledged limitation. The Tier Safety Gate and Sliding Window are built directly from this finding — the safe token budget for each model tier is derived from the degradation pattern Liu et al. document, not chosen arbitrarily.

2.3 Existing Adaptive RAG Systems

System	What It Does	What It Does Not Do
--------	--------------	---------------------

Adaptive-RAG (2024)	Adapts retrieval strategy by query complexity	Fixed k within each strategy
Self-RAG (2024)	Adapts decision to retrieve via self-reflection	Requires LLM fine-tuning. Fixed k when retrieving.
FLARE (2023)	Triggers retrieval on generation uncertainty	Fixed k when triggered. No latency guarantee.
RankRAG (2024)	Fine-tuned LLM for ranking and generation	No latency governance. Fixed k output.
CAR (2024)	Cluster-based adaptive retrieval depth	No latency governor. No sliding window.
Ada-ef (2026)	Adaptive efSearch for HNSW per query	HNSW only. Does not predict k.
LARA (ours)	Adaptive k, quality gate, latency proof, context safety	Full pipeline — all four problems addressed.

2.4 Two-Stage Retrieve-then-Rerank

Cross-encoder rerankers process query-document pairs jointly through a transformer network, producing relevance scores that capture semantic interaction with higher fidelity than bi-encoder similarity (Reimers and Gurevych, 2019). The standard production recommendation is to retrieve 50-100 candidates with a bi-encoder and rerank to a fixed top 5-10 with a cross-encoder. LARA uses this two-stage pattern but replaces the fixed top-k output from the cross-encoder with a score threshold filter — making the final retrieval count a function of actual relevance distribution rather than a developer-specified constant.

Key Distinction: Every existing adaptive RAG system either adapts whether to retrieve, or adapts the retrieval strategy, but still applies a fixed k when retrieval actually happens. LARA is the first system to make k itself a dynamic output of the pipeline — determined by actual relevance, constrained by a mathematical latency guarantee, and delivered to the LLM in tier-safe batches derived from Liu et al.'s context degradation findings.

3. The LARA Architecture

LARA processes every query through four phases. Each phase has three things: why it exists, what it does precisely, and what its output is.

3.1 Phase 1 and 2 — ANN Indexing and Candidate Capture

Why this phase exists. Running a cross-encoder across an entire corpus of even 100,000 documents produces catastrophic latency — cross-encoders scale at $O(N)$. We need to narrow the candidate pool to a feasible size before quality gating begins.

What it does. During indexing (Phase 1), the corpus is chunked at developer-specified character size — typically 1,000 characters with 200-character overlap. Each chunk is vectorised using a bi-encoder (BAAI/bge-small-en-v1.5) and stored in an ANN index using IVF or HNSW structure. This happens once at build time. During retrieval (Phase 2), the user query is vectorised and the ANN index returns a candidate pool — chunks geometrically close to the query vector in embedding space. In the current implementation this pool is approximately 200 chunks, retrieved in $O(\log N)$ time. The ML extension in Section 5 makes this pool size adaptive as well.

Output. A candidate pool of approximately 200 chunks that are geometrically close to the query. Some will be semantically irrelevant despite being close in embedding space. Phase 3 fixes this.

3.2 Phase 3 — The Quality Gate

Why this phase exists. Vector similarity search is approximate. A chunk can be close to the query embedding but factually irrelevant — vibes over facts. The cross-encoder sees the full text of query and document jointly, catching semantic relevance that bi-encoder similarity misses.

What it does. LARA applies a cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) to every chunk in the candidate pool, producing a relevance score $S(c)$ between 0 and 1 for each chunk c . Any chunk scoring below $\theta = 0.3$ is immediately eliminated. What remains is `Dynamic_K`.

Quality Gate — Dynamic K Definition

$\text{Dynamic_K} = \{ c \in \text{Candidate_Pool} \mid S(c) \geq \theta \}$, $\theta = 0.3$ `Dynamic_K` is then sorted by descending score. Highest-confidence chunks are always processed first.

The threshold $\theta = 0.3$ was selected empirically for the ms-marco-MiniLM cross-encoder family. Chunks scoring below 0.3 consistently failed to contribute positively to answer generation and frequently introduced contradictory information. This threshold is model-specific — a different

cross-encoder requires recalibration.

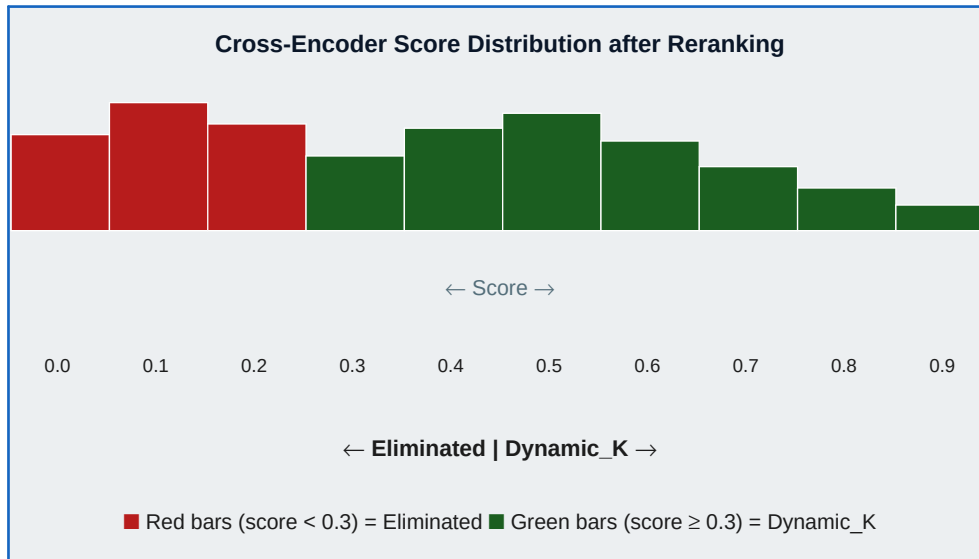


Figure 2: Score distribution after cross-encoder reranking. Red bars (score < 0.3) are eliminated. Green bars form Dynamic_K.

3.3 Phase 4 — The Latency Governor

Why this phase exists. Dynamic_K may be too large to process within the developer's acceptable response time. We need a mechanism that guarantees the system responds within budget regardless of corpus size or query complexity. This is not a heuristic — it is a mathematical guarantee.

Variable	Definition
L_afford	Total affordable latency — developer-specified (e.g. 10 seconds)
T_rerank	Time consumed by Phases 1, 2, and 3 — measured at runtime
Lat_chunk	Time to process one chunk = chunk_tokens × time_per_token
L_budget	Remaining latency for LLM = L_afford – T_rerank
Exceeded	$\max(0, \text{Dynamic_K} \times \text{Lat_chunk} - \text{L_budget})$
Reduce_Count	$\text{ceil}(\text{Exceeded} / \text{Lat_chunk})$ — chunks to remove
Send_List	Dynamic_K with lowest-scored chunks removed until budget is met

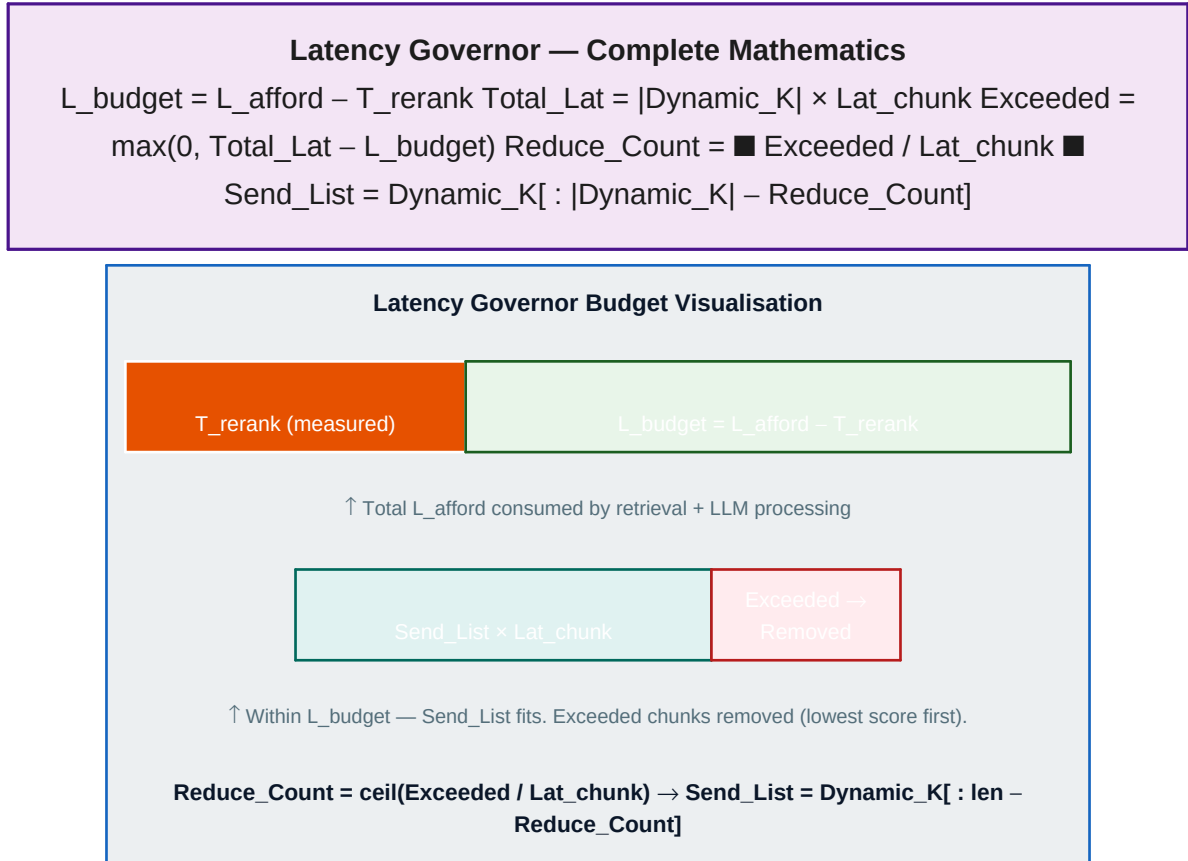
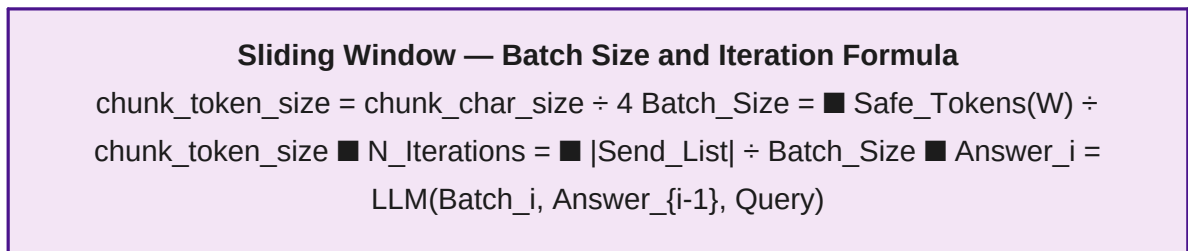


Figure 3: Latency Governor budget visualisation. T_rerank consumed first. Remaining L_budget governs $Send_List$ size.

3.4 Phase 5 — Tier Safety Gate and Sliding Window Refinement

Why this phase exists. The Latency Governor ensures $Send_List$ can be processed within time. But time alone does not guarantee safe LLM processing. Even a latency-compliant $Send_List$ can exceed the safe context limit for the model being used, triggering middle context loss. Phase 5 solves this independently — latency governor owns time, tier table owns context safety.

How batch size is derived. The user specifies $chunk_char_size$ during vectorisation (e.g. 1,000 characters). This converts to token count using the standard approximation of 4 characters per token. The model in use determines $Safe_Tokens(W)$ from the tier table. Batch size follows directly.



Worked example. $Send_List$ has 50 chunks. $chunk_char_size=1000$ so $chunk_token_size=250$. Model is Ollama 7B (Layer 1, $Safe_Tokens=3,000$). $Batch_Size = 3000 \div 250 = 12$ chunks.

$N_Iterations = \text{ceil}(50 \div 12) = 5$. The model never sees more than 3,000 tokens in one call. Running answer accumulates across 5 iterations.

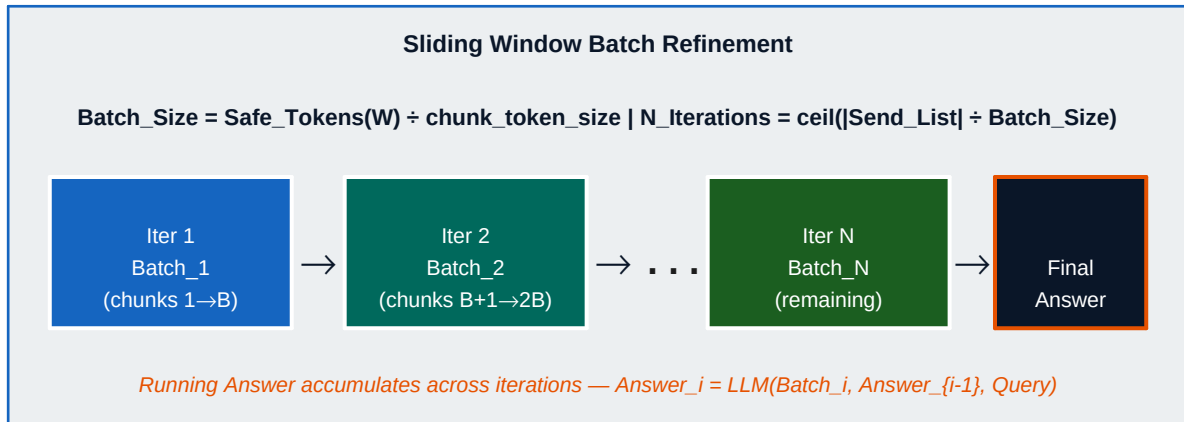


Figure 4: Sliding Window Batch Refinement. Batch size from tier safety limit. Running answer accumulates across iterations.

4. Theoretical Analysis

4.1 $O(\log N)$ Scaling Argument

A naive quality-gated RAG system would run the cross-encoder across the entire corpus. For N documents, cross-encoder complexity is $O(N)$. At $N=1,000,000$ this is catastrophic. LARA uses ANN indexing to reduce Phase 2 to $O(\log N)$. HNSW achieves this through layered graph traversal — search starts at a coarse top layer and descends to finer layers, narrowing toward the query neighbourhood. IVF achieves sub-linear search by computing distances only to n_{probe} cluster centroids, then searching only vectors within those clusters. In both cases, the cross-encoder in Phase 3 runs on approximately 200 candidates — a constant — rather than the full corpus. The result is $O(\log N)$ candidate capture followed by $O(1)$ quality gating, giving LARA the accuracy of a cross-encoder with the speed of a bi-encoder.

4.2 Latency Governor Correctness Proof

Latency Governor — Formal Correctness Proof

Theorem: For any `Send_List` produced by the Latency Governor, $|\text{Send_List}| \times \text{Lat_chunk} \leq \text{L_budget}$ Proof — let $n = |\text{Dynamic_K}|$, $r = \text{Reduce_Count}$ Case 1: $n \times \text{Lat_chunk} \leq \text{L_budget}$ Exceeded = 0, $r = 0$, `Send_List` = `Dynamic_K` Therefore: $n \times \text{Lat_chunk} \leq \text{L_budget}$ ✓ Case 2: $n \times \text{Lat_chunk} > \text{L_budget}$ $r = \lceil (n \times \text{Lat_chunk} - \text{L_budget}) / \text{Lat_chunk} \rceil$ $(n - r) \times \text{Lat_chunk} \leq n \times \text{Lat_chunk} - (n \times \text{Lat_chunk} - \text{L_budget}) = \text{L_budget}$ ✓ The ceiling function ensures we never undercount the required reduction. Budget compliance is guaranteed in all cases. ■

4.3 Middle Context Loss Prevention

Liu et al. (2024) show that performance degrades when relevant content is in the middle of a long context. The sliding window prevents this structurally: each batch contains at most $\text{Batch_Size} = \text{Safe_Tokens}(W) \div \text{chunk_token_size}$ chunks. Total tokens in any single LLM call never exceed $\text{Safe_Tokens}(W)$. The model therefore always sees relevant content near the start of its context within each batch. Running memory from previous batches is prepended as a short refined answer, not as the full previous batch, keeping each call well within the safe operating zone.

4.4 Deriving the Safety Tier Table

The tier table is not arbitrary. Liu et al. (2024) show that safe utilisation decreases as a proportion of total context window as window size grows — a model with a large context window suffers more severe middle context loss at the same utilisation percentage than a model with a small window. Using this decreasing utilisation pattern as the basis, I derive four tiers anchored at the four most representative model classes in production as of 2025.

Layer	Window	Safe_Tokens	Utilisation	Derivation Basis
Layer 1	4K–6K	3,000	75%	Local (Ollama 7B, Gemma). Small context. Liu et al. show minimal degradation at 75%.
Layer 2	7K–12K	5,000	62.5%	Cloud-lite (GPT-3.5-Turbo). Degradation begins. Safety margin widened.
Layer 3	~128K	30,000	23%	Pro cloud (GPT-4o, Claude 3). Severe middle degradation per Liu et al.
Layer 4	1.5M+	300,000	20%	Long-ctx (Gemini 1.5 Pro). Conservative margin for extreme context.

Continuous Safe_Tokens Formula

For models not in the four tiers: $\text{Safe_Tokens}(W) = W \times f(W)$ $f(W)$ is monotonically decreasing, fitted log-linearly from the four anchors: $f(4K) = 0.75$, $f(8K) = 0.625$, $f(128K) = 0.23$, $f(1.5M) = 0.20$ $f(W) = a - b \times \log_{10}(W)$ where a, b fitted from these four points

The four specific Safe_Tokens values are my empirically derived operational choices, consistent with and motivated by Liu et al. but not directly read from their paper. A larger study across more models and context lengths would refine them. This is acknowledged as a limitation in Section 8.

5. ML Extension — Index-Structure-Aware K Prediction

Status: The Index-Structure-Aware K Prediction is a proposed extension. The theoretical motivation is complete and the architecture is fully specified. Training data generation, model training, and benchmark evaluation are planned for the companion empirical paper. I present it here because the core insight — using index structure rather than random sampling — is novel and architecturally integral to LARA's completeness.

5.1 The Problem With Fixed Candidate Pool Size

LARA currently retrieves approximately 200 candidates in Phase 2. This was chosen as a reasonable default — large enough for most queries, small enough for the cross-encoder. But 200 is itself a fixed constant. It has the same fundamental problem as fixed $k=4$, just at a different point in the pipeline.

A corpus of 50 total chunks: retrieving 200 candidates means retrieving everything multiple times — wasteful and meaningless. A corpus of 10 million chunks on a specific technical query: 200 candidates may not capture all relevant chunks if they are scattered across distant embedding regions. The optimal candidate pool depends on the query and corpus structure. It should be predicted, not fixed.

The naive solution is random sampling — draw some samples from the corpus and use sample statistics to estimate the neighbourhood size. But this introduces a new arbitrary constant: how many samples? 50 is too few for a large corpus. 10,000 adds significant latency. There is no principled fixed number. The solution: do not sample randomly. Read the index structure itself.

5.2 Index Structure as Distribution Proxy

When a vector index is built, it does not just store vectors — it organises them into a structure that encodes information about their distribution. The index structure is a compressed, already-computed summary of where vectors are in embedding space and how they relate to each other. Reading this structure costs nothing extra — it already exists. Using it to estimate the relevant neighbourhood size for a specific query is the core insight of the ML extension.

Index Structure as Distribution Proxy

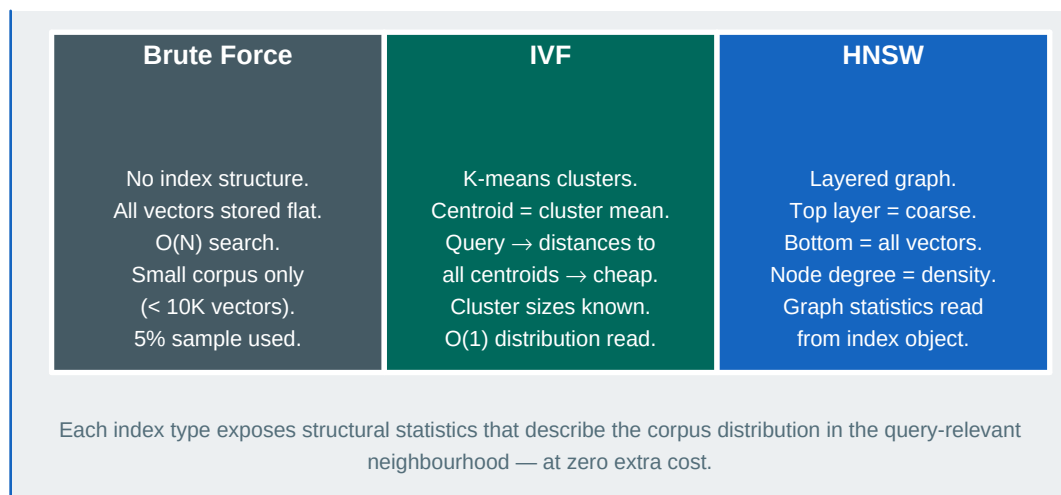


Figure 5: Three index types and the structural signals they expose. Each provides distribution information about the corpus at zero extra cost.

5.3 How Each Index Type Is Read

For IVF — the clearest case:

IVF clusters all vectors using k-means. Each cluster has a centroid stored in the index. Computing distances from the query to all centroids is cheap — typically 100 to 1,000 computations regardless of corpus size. From these distances the ML model reads: which centroids are within threshold distance (relevant clusters), how many vectors are in each relevant cluster (cluster sizes stored in index), and how spread the relevant clusters are. If 3 clusters are within threshold distance and each has 80 vectors, the model predicts $k \approx 240$ as the candidate pool. No random sampling. The index already computed this.

For HNSW — graph statistics:

HNSW builds a layered graph. The top layer has few nodes with long-range connections. Lower layers have all nodes with local connections. The ML model reads three signals from the HNSW index object: the number of nodes at the entry layer within geometric threshold of the query (regional density estimate), the average degree (number of connections) of nodes in the relevant region (high degree = dense packing = more candidates), and the number of graph layers (encodes corpus size). These translate directly to a candidate pool size estimate.

For Brute Force — proportional sampling:

Brute Force has no index structure. But Brute Force is only appropriate for small corpora — under roughly 10,000 vectors. At this size, 5% proportional sampling costs at most 500 distance computations — fast and feasible. This is the one case where sampling is acceptable, because the corpus is small enough that any sampling count is manageable.

5.4 Model Architecture and Training

The full ML extension requires two components working together.

Database Type Classifier:

Classifies the index as HNSW, IVF, or Brute Force. In practice this is rule-based rather than learned — the index object's class attributes identify the type unambiguously. FAISS IVF indexes have a quantizer attribute. HNSW indexes have an hnsw attribute with level structure. Flat indexes have neither. A three-line conditional achieves this without ML.

Candidate Pool Size Regressor:

The learned component. Takes as input the index structure statistics described above — centroid distances and cluster sizes for IVF, graph degree and layer statistics for HNSW, sample statistics for Brute Force — combined with a compressed query embedding. Outputs predicted candidate pool size k^* .

Training data is generated by running LARA on NaturalQuestions and HotpotQA across many corpora and recording, for each query, the smallest candidate pool that captured all chunks scoring above 0.3 on the cross-encoder. This becomes the ground truth k^* for each (query, corpus) pair. The regressor learns to predict k^* from index structure statistics without running the cross-encoder first.

Feature	Description	Index Type
Query embedding	128-dim compressed (PCA)	All types
Centroid distances	Query to all IVF centroids	IVF only
Cluster sizes	Vectors per relevant cluster	IVF only
Cluster spread	Variance of centroid distances	IVF only
Graph layer stats	Node count per HNSW layer	HNSW only
Node degree	Avg connections in neighbourhood	HNSW only
Corpus size	Total vectors in index	All types
Sample distances	5% proportional sample	Brute Force only

6. Implementation

LARA is implemented as the retrieval backend of the NPM-RAG-API-Framework and is accessible through the npmai Python library, deployed on HuggingFace Spaces with Supabase for persistent vector storage.

Component	Specification	Role
Cross-Encoder	cross-encoder/ms-marco-MiniLM-L-6-v2	Quality Gate — Phase 3
Bi-Encoder	BAAI/bge-small-en-v1.5	Vectorisation — Phase 1
Vector Store	FAISS with IVF/HNSW indexing	ANN Capture — Phase 2
Chunk Size	1,000 characters, 200-char overlap	Indexing
API	FastAPI on HuggingFace Spaces	Deployment
Database	Supabase (PostgreSQL + pgvector)	Persistent storage
Default LLM	Ollama 7B local	Generation — Phase 5

lara_retrieve — Phases 2, 3 and 4

```
# Phases 2, 3, 4 – Candidate Capture, Quality Gate, Latency Governor
def lara_retrieve(query, vectordb, L_afford=10.0,
                 lat_chunk=0.1, threshold=0.3, pool=200):
    t0 = time.time()
    # Phase 2: ANN Candidate Capture – O(log N)
    candidates = vectordb.similarity_search(query, k=pool)
    # Phase 3: Quality Gate – Cross-Encoder + 0.3 Threshold
    reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
    scores = reranker.predict([(query,d.page_content) for d in candidates])
    dynamic_k = sorted([(d,s) for d,s in zip(candidates,scores)
                       if s >= threshold], key=lambda x:x[1], reverse=True)
    dynamic_k = [d for d,_ in dynamic_k]
    T_rerank = time.time() - t0
    # Phase 4: Latency Governor
    L_budget = L_afford - T_rerank
    total_lat = len(dynamic_k) * lat_chunk
    if total_lat > L_budget:
        reduce = math.ceil((total_lat - L_budget) / lat_chunk)
        return dynamic_k[:len(dynamic_k) - reduce]
    return dynamic_k
```

lara_refine — Phase 5 Sliding Window


```
# Phase 5 – Tier Safety Gate + Sliding Window Refinement
def lara_refine(send_list, query, llm,
               chunk_char_size=1000, safe_tokens=3000):
    chunk_tokens = chunk_char_size // 4      # 4 chars ~ 1 token
    batch_size   = safe_tokens // chunk_tokens
    n_iter       = math.ceil(len(send_list) / batch_size)
    answer = ""
    for i in range(n_iter):
        batch = send_list[i*batch_size : (i+1)*batch_size]
        context = "\n---\n".join([d.page_content for d in batch])
        prompt = (f"Context:\n{context}\n"
                  f"Current Answer: {answer}\n"
                  f"Question: {query}\nRefined Answer:")
        answer = llm.invoke(prompt)
    return answer
```

7. Experimental Setup

Honest statement on results: Full benchmark results are pending. This paper presents the architectural design, theoretical proofs, and deployment evidence. A companion empirical paper will report complete results. The experimental protocol below is specified in full for reproducibility — anyone can run these experiments against the open-source implementation.

7.1 Datasets

Dataset	Description	What It Tests
NaturalQuestions (Kwiatkowski et al., 2019)	Open-domain QA, Wikipedia knowledge source	Single-hop factual retrieval. Tests noise reduction.
HotpotQA (Yang et al., 2018)	Multi-hop reasoning across multiple documents	Tests sliding window evidence accumulation.
NPMAI Custom Corpus	50 technical documents: ML papers, API docs	Domain-specific deployment validation.

7.2 Baselines

Baseline	Description	Purpose
Standard RAG k=4	LangChain default pipeline	Lower bound on recall
Standard RAG k=10	Fixed k=10 for higher recall	Naive scaling comparison
Naive Two-Stage	ANN + fixed top-10 cross-encoder	Isolates threshold contribution
LARA (ours)	Full four-phase pipeline	Primary system

7.3 Metrics

Metric	Definition
Exact Match (EM)	Does output exactly match gold answer
F1 Score	Token-level precision-recall balance
End-to-End Latency	Total time from query to final answer
Context Noise Rate	Proportion of Send_List chunks with score < 0.3
Dynamic K Size	Average chunks surviving quality gate across queries

8. Discussion

8.1 Why Training-Free Matters

DynamicRAG and Self-RAG are powerful systems. But they require either fine-tuning the language model or training a separate controller on labelled data. This is inaccessible to most independent developers and organisations deploying RAG in resource-constrained environments — which is LARA's primary target. LARA's quality gate, latency governor, and sliding window require only a cross-encoder (freely available via Sentence Transformers) and standard FAISS or HNSW libraries. No training. No labelled data beyond your corpus.

8.2 The Deployment Evidence

The npmai package has over 700,000 total installations with consistent daily downloads. This is not benchmark evidence — it is adoption evidence. Real developers found the system useful enough to install and integrate. It does not substitute for controlled evaluation but it demonstrates that the architecture is deployable, functional, and solves a real problem practitioners experience. The formal benchmarks will quantify what this deployment evidence suggests qualitatively.

8.3 Honest Limitations

- **The 0.3 threshold is model-specific.** Calibrated for ms-marco-MiniLM. A different cross-encoder requires recalibration.
- **The tier table `Safe_Tokens` values are empirically derived.** Grounded in Liu et al. (2024) but the specific numbers are operational choices. A larger study would refine them.
- **The ML extension is not yet trained or benchmarked.** The architecture is specified and novel. Training and evaluation are future work.
- **`Lat_chunk` is developer-estimated.** The guarantee holds for the estimated value. Real hardware variance may cause occasional budget overruns if the estimate is significantly off.
- **4-chars-per-token is an approximation.** Different tokenizers and languages have different ratios. Production systems should use the actual tokenizer for precise batch size calculation.

9. Conclusion

The fixed-k problem in RAG is not a minor inefficiency. It is a fundamental design flaw that produces noise injection, context overflow, and middle context loss across all standard RAG deployments. LARA addresses this through four principled phases: ANN candidate capture in $O(\log N)$, cross-encoder quality gating with a score threshold that makes retrieval count a function of actual relevance, a Latency Governor with a formal mathematical correctness proof, and a Tier Safety Gate with Sliding Window Refinement that derives batch size from Liu et al.'s middle context loss findings rather than an arbitrary constant.

Every design decision in LARA has a basis — either a formal derivation or a grounded empirical finding from the literature. Nothing in the pipeline is chosen arbitrarily. This is the property that standard RAG's fixed-k fundamentally lacks.

The ML extension — Index-Structure-Aware K Prediction — extends this principle to the candidate pool size itself, using the internal structure of the vector index as a distribution proxy rather than a fixed constant or random sampling. This is, to my knowledge, a novel approach that no existing system has implemented.

LARA is deployed. It works. The benchmarks will quantify what the 700,000 installations suggest qualitatively. The companion empirical paper will complete the picture. For now, the architecture is here, the proofs are here, and anyone who wants to build on or benchmark against it can do so from the open-source repository.

Repository: github.com/sonuramashishnpm/NPM-Rag-API-Framework

Package: `pip install npmai`

Platform: npmai.netlify.app

References

- [1] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR 2024*.
- [2] Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024). RankRAG: Unifying context ranking with retrieval-augmented generation in LLMs. *NeurIPS 2024*.
- [3] Jeong, S., Baek, J., Cho, S., Hwang, S. J., and Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv:2403.14403*.
- [4] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., and Neubig, G. (2023). Active retrieval augmented generation. *arXiv:2305.06983*.
- [5] Kumar, S. (2025). NPM-RAG-API-Framework. NPMAI ECOSYSTEM. github.com/sonuramashishnpm/NPM-Rag-API-Framework
- [6] Kumar, S. (2025). npmai Python package. PyPI. pypi.org/project/npmai
- [7] Kwiatkowski, T. and others. (2019). Natural Questions: A benchmark for question answering research. *TACL*, 7, 452–466.
- [8] Lewis, P. and others. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- [9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *TACL*, 12, 157–173.
- [10] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP 2019*.
- [11] Sun, J. and others. (2025). DynamicRAG: Leveraging outputs as feedback for adaptive retrieval-augmented generation. *arXiv:2025*.
- [12] Taguchi, H. and others. (2025). Adaptive-k: Efficient context selection for long-context QA. *EMNLP 2025*.
- [13] Yang, Z. and others. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. *EMNLP 2018*.
- [14] Zhang, C. and Miller, R. J. (2026). Distribution-Aware Exploration for Adaptive HNSW Search. *SIGMOD 2026*. *arXiv:2512.06636*.